SIMATS ENGINEERING

Assessment Tool 1 – Application-Based Questions

Course: Internet Programming (CSA43) | CO-1 | Total Marks: 30

Name / Reg No: Vijayan / _____

Question 1: Online Symposium Portal

The given form works in a browser, but it is only a bare fragment. It has no document structure, no labels and no semantic wrappers, so it is not really standards-compliant HTML5.

1. Missing HTML5 semantic elements

- **Document skeleton:** <!DOCTYPE html>, <html lang="en">, <head> with <meta charset> and <title>, and <body>. Without the DOCTYPE the browser falls into quirks mode.
- **<header>:** for the portal title or logo, e.g. "National Symposium 2026 – Registration".
- **<main> and <section>:** to hold the registration area as the main content of the page.
- **<fieldset> and <legend>:** to group the related form controls and give the group a name.
- **<label>:** every <input> needs one. This is the biggest gap in the snippet, because screen readers and users who tap the field cannot tell what each box is for.
- **<footer>:** contact details, copyright, help email.

2. Standards-compliant structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>National Symposium 2026</h1>
  </header>

  <main>
    <section id="registration">
      <h2>Student Registration</h2>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Your details</legend>

          <label for="name">Full name</label>
          <input type="text" id="name" name="name"
```

```
                required autocomplete="name">

          <label for="email">Email address</label>
          <input type="email" id="email" name="email"
                 required autocomplete="email">

          <button type="submit">Submit</button>
        </fieldset>
      </form>
    </section>
  </main>

  <footer>
    <p>Contact: symposium@college.edu</p>
  </footer>
</body>
</html>
```

Changes made: full document skeleton, semantic wrappers, id/for pairs linking labels to inputs, the required and autocomplete attributes, and an explicit type="submit" on the button (a <button> with no type is a submit button by default, but stating it avoids confusion).

Web page created for Question 1: the corrected code above was saved as q1_symposium.html and opened in a browser. The screenshot below is the output.

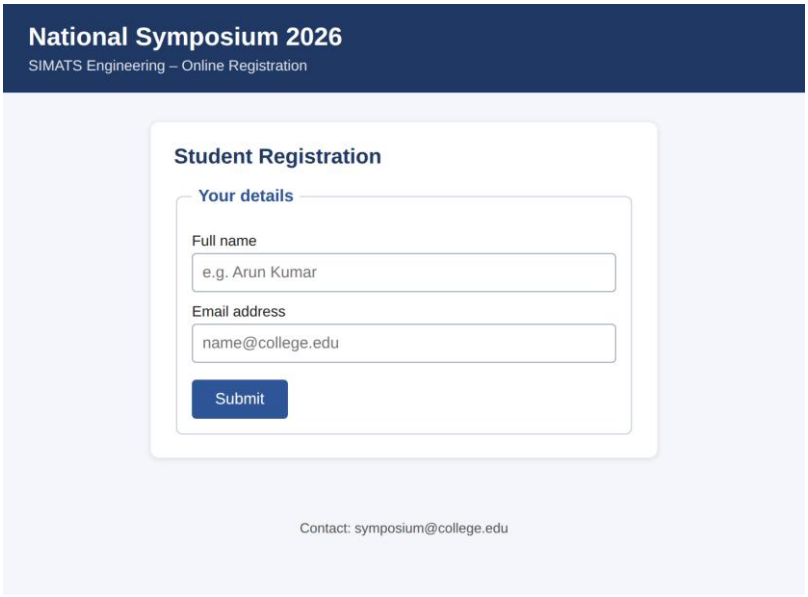


Figure 1: Output of the HTML web page created for Question 1 (q1_symposium.html)

3. HTTP request generated on submission

Since the method is POST, the browser puts the form data in the request body, not in the URL. For example, if a student enters "Arun Kumar" and "arun@example.com", the request looks roughly like this:

```
POST /register HTTP/1.1
Host: www.college.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 39
User-Agent: Mozilla/5.0 ...
Cookie: sessionid=abc123

name=Arun+Kumar&email=arun%40example.com
```

Part	What it tells us
Request line	POST is the method, /register is the path taken from the form action, HTTP/1.1 is the protocol version.
Host header	Which website on the server the request is meant for.
Content-Type	application/x-www-form-urlencoded is the default encoding. Data is sent as name=value pairs joined by &, with spaces turned into + and special characters percent-encoded (@ becomes %40).
Content-Length	Size of the body in bytes, so the server knows where it ends.
Body	The actual form data. Only fields that have a name attribute are sent, which is why the name attribute on each input matters.

4. HTTP response cycle from server to browser

1. **Connection:** the browser resolves the domain through DNS, opens a TCP connection and, for HTTPS, completes the TLS handshake. Then it sends the request above.
2. **Server processing:** the web server routes /register to the application code. The code reads the body, validates the name and email, and stores the record in the database.
3. **Response:** the server sends a status line, headers and optionally a body. A good pattern here is Post/Redirect/Get, where the server answers with 302 Found and a Location: /thank-you header.
4. **Browser follows the redirect:** it makes a fresh GET request to /thank-you, and the server returns 200 OK with the confirmation HTML (Content-Type: text/html).
5. **Rendering:** the browser parses the HTML into the DOM, requests any extra CSS, JS and images, and paints the confirmation page.

If something goes wrong the status code changes: 400 Bad Request for invalid input, 409 if the email is already registered, 500 Internal Server Error if the server code fails. The redirect step is useful because pressing refresh on the thank-you page then does not re-submit the form and create a duplicate registration.

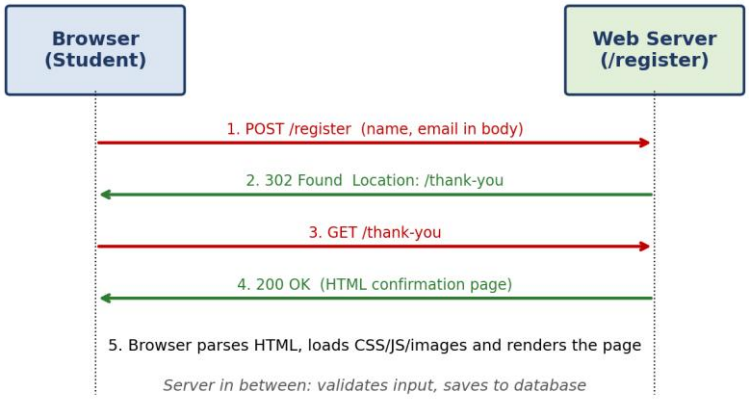


Figure 2: HTTP request-response cycle for the registration form (Post/Redirect/Get)

5. Usability and accessibility improvements

- **Labels and hints:** visible labels tied to inputs, plus placeholder text such as "name@college.edu" as a hint only, never as a replacement for the label.
- **Validation:** use required, type="email" and pattern for instant feedback in the browser, but always validate again on the server because client checks can be bypassed.
- **Error messages:** show clear text next to the field and link it with aria-describedby; use role="alert" so screen readers announce it.
- **Keyboard and focus:** logical tab order, a visible focus outline, and large tap targets for mobile users.
- **Contrast and responsiveness:** enough colour contrast (WCAG AA, 4.5:1 for text), relative font sizes, and the viewport meta tag so the form works on phones.
- **Confirmation:** show a clear success message or send a confirmation email after registering.
- **Security basics:** serve the page over HTTPS and add a CSRF token to the form.
- **Extra fields:** college name, department and event choice using <select>, with autocomplete attributes so the browser can fill the details.

Question 2: Cross-Browser Compatibility Issue

1. Improper nesting and missing closing tags

- <h1> is opened but never closed (no </h1>).
- is opened and never closed (no). HTML5 lets a browser infer the end of a in some cases, but relying on that is bad practice and it is plainly an error in XHTML.
- **Improper nesting:** because </h1> is missing, the ends up inside the heading. A paragraph is block-level (flow) content and an <h1> may only contain inline (phrasing) content, so this nesting is invalid.
- The page has no <!DOCTYPE html>, no lang attribute on <html>, and no <meta charset>.

2. How different browsers may interpret the structure

The missing DOCTYPE is the first problem. Without it, browsers switch to quirks mode, an old compatibility mode where box model, line height and table sizing rules differ between browsers. That alone can make the same page look different in Chrome, Firefox, Safari and older Internet Explorer.

Next, the unclosed tags. Modern browsers use HTML5 error-recovery rules, so they will not crash, but they have to guess. In most of them the text "About us" ends up inside the <h1>, so it is displayed with heading size and bold weight even though the author meant a normal paragraph. Older browsers followed their own, slightly different recovery logic, which produced different DOM trees and therefore different spacing and styling. Finally, if the page were served as XHTML (application/xhtml+xml), the stricter XML parser would simply refuse to render it and show a parse error, since the document is not well-formed.

In short, the browser is not rendering what the developer wrote, it is rendering its own guess at it, and each browser guesses a little differently. That is the root cause of the inconsistent layout.

3. Missing semantic elements

- **<header>** for the site title and branding area.
- **<nav>** for the navigation menu (not present yet but a company site needs one).
- **<main>** for the primary content of the page.
- **<section>** (or <article>) for the "About us" block, with its own <h2> heading.
- **<footer>** for contact and copyright information.
- Also, the is just a generic container and gives no meaning to browsers, search engines or screen readers.

4. Corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>

  <main>
    <section>
      <h2>About Us</h2>
      <p>About us</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Company</p>
  </footer>
</body>
</html>
```

This version closes every tag, nests elements properly (heading and paragraph are siblings, not parent and child), and adds the DOCTYPE, language, charset and semantic landmarks. It should pass the W3C Markup Validator without errors.

Web page created for Question 2: the corrected structure was saved as q2_company.html (a small stylesheet and a nav bar were added so the output looks like a real company page). Output in the browser:

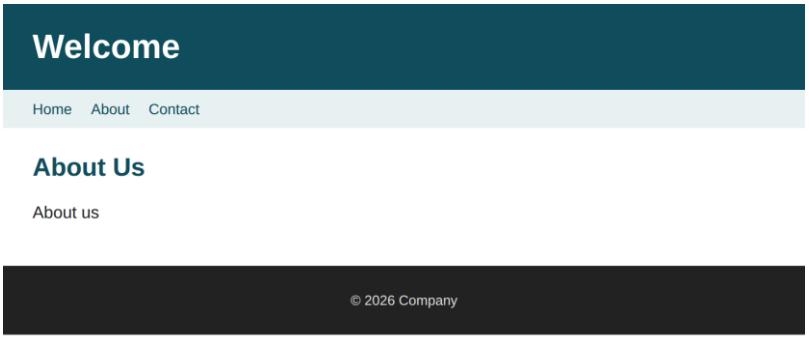


Figure 3: Output of the HTML web page created for Question 2 (q2_company.html)

5. Techniques to ensure cross-browser compatibility

- **Always declare the DOCTYPE** so every browser uses standards mode.
- **Validate the code** with the W3C Markup Validator and CSS Validator before testing.
- **Use a CSS reset or normalize.css** to remove the default margin and padding differences between browsers.
- **Prefer standard features** and check support on caniuse.com. Use feature detection (@supports in CSS, or Modernizr in JavaScript) rather than checking which browser it is.
- **Progressive enhancement and fallbacks:** build a working basic version first, then add newer features on top. Provide a fallback for things like CSS Grid.
- **Vendor prefixes:** let a tool such as Autoprefixer add them automatically.
- **Responsive design:** viewport meta tag, flexible units (%, rem), media queries.
- **Test on real browsers and devices:** Chrome, Firefox, Safari, Edge, plus mobile, using DevTools device mode or services like BrowserStack.
- **Avoid deprecated tags** (<font>,) and browser-specific hacks.

Question 3: Form Submission Failure

1. Issues in the form configuration

1. **The button has type="button".** This kind of button does nothing by default. It does not submit the form. This is the main bug.
2. **method="GET" with a password field.** If the form did submit, the username and password would be appended to the URL.
3. **No <label> elements** and no id attributes, so the form is not accessible and clicking a label will not focus the input.
4. **No validation attributes** (required) and no autocomplete hints, so empty submissions are possible and password managers may not work properly.
5. **No JavaScript handler** attached to the button, so there is nothing else that could send the data either.

2. Why the HTTP request is not triggered

A form is submitted by a submit button (<button type="submit">, <input type="submit">) or by a script calling form.submit(). Here the button is type="button", which is a plain push button with no default behaviour, and no JavaScript is wired to it. So clicking it just does nothing and the browser never builds a request.

Pressing Enter inside a text box will not rescue it either. HTML only allows this "implicit submission" when the form has a submit button, or when it has just one text-type field. This form has two fields and no submit button, so Enter is ignored too. That is why the application looks like it is silently failing, there is no error, simply no request in the Network tab.

3. Role of the GET method in this scenario

GET is meant for retrieving data and it sends form values in the URL query string. If the button were fixed but the method left as GET, the browser would request something like:

```
GET /submit?username=vijay&password=MyPass123 HTTP/1.1
Host: www.example.com
```

That is a real problem for a login form:

- The password is visible in the address bar and can be seen by anyone looking at the screen.
- It gets saved in browser history, bookmarks and autofill suggestions.
- It is written to web server, proxy and CDN logs in plain text.
- It can leak to other sites through the Referer header.
- URLs have length limits and GET responses can be cached, neither of which suits credentials.

GET is supposed to be safe and idempotent (it should not change anything on the server). A login creates a session, so it belongs with POST.

4. Corrected implementation

```
<form action="/submit" method="POST">
  <label for="username">Username</label>
  <input type="text" id="username" name="username"
         required autocomplete="username">

  <label for="password">Password</label>
  <input type="password" id="password" name="password"
         required autocomplete="current-password">

  <button type="submit">Login</button>
</form>
```

Fixes: the button is now type="submit" so the form actually sends, the method is POST so the credentials go in the request body, labels are linked through for/id, and required plus autocomplete are added.

Web page created for Question 3: the corrected login form was saved as q3_login.html. Now the Login button is a real submit button, so clicking it sends a POST request to /submit. Output in the browser:

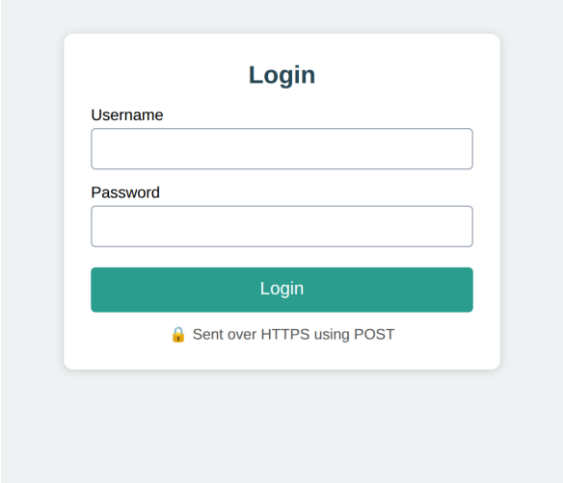


Figure 4: Output of the HTML web page created for Question 3 (q3_login.html)

5. Improvements for secure data transmission

- **Use HTTPS (TLS) everywhere,** with HSTS to force it. Even POST data is readable on the network without encryption.
- **Always use POST** for credentials, never GET.
- **Hash passwords on the server** with bcrypt or Argon2 and a salt. Never store them in plain text.
- **CSRF protection:** include a hidden anti-CSRF token in the form and verify it on the server.
- **Secure cookies:** set the session cookie with Secure, HttpOnly and SameSite flags.
- **Server-side validation and rate limiting** to block brute-force attempts, with account lockout or CAPTCHA after repeated failures.
- **Prevent caching of sensitive responses** using Cache-Control: no-store, and do not log passwords.
- **Add multi-factor authentication** where possible, and a Content Security Policy to reduce XSS risk.